

Name:	Muhammad Hasnat Fakhar
Class/Section:	BSCSev-F23-C
Reg ID:	232436
Subject:	Parallel and Distributing Computing
Task:	LAB TASK 09

LAB TASK 01

Code

```
using System;
using System.Diagnostics;
using MPI;

0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        using (new MPI.Environment(ref args))
        {
            Intracommunicator world = Communicator.world;
            int rank = world.Rank;
            int[] sizes = { 0, 1, 8, 64, 256, 1024, 4096, 16384, 65536, 262144 };
            int iterations = 100;

            if (rank == 0) Console.WriteLine("Size (Bytes) | Mean RTT (us)");

            foreach (int size in sizes)
            {
                // Min 1 byte allocate karenge taake 0-byte par crash na ho
                int actualSize = Math.Max(1, size);
                byte[] buffer = new byte[actualSize];
                byte[] recvBuffer = new byte[actualSize];

                world.Barrier(); // Sync before timing

                Stopwatch sw = Stopwatch.StartNew();
                for (int i = 0; i < iterations; i++)
                {
                    if (rank == 0)
                    {
                        world.Send<byte>(buffer, 1, 0);
                        world.Receive<byte>(1, 0, ref recvBuffer);
                    }
                    else if (rank == 1)
                    {
                        world.Receive<byte>(0, 0, ref recvBuffer);
                        world.Send<byte>(buffer, 0, 0);
                    }
                }
                sw.Stop();

                if (rank == 0)
                {
                    double rttMicrosec = (sw.Elapsed.TotalMilliseconds * 1000) / iterations;
                    Console.WriteLine($"{size,-12} | {rttMicrosec:F2}");
                }
            }
```

OUTPUT

```
Problems  Output  Debug Console  Terminal  Ports

PS E:\Lab-09\Task1-PingPong> cd E:\Lab-09\Task1-PingPong
>> dotnet build -c Release
>> mpiexec -n 2 .\bin\Release\net48\Task1-PingPong.exe
Restore complete (0.5s)
  Task1-PingPong net48 win-x86 succeeded (0.3s) → bin\Release\net48\Task1-PingPong.e

Build succeeded in 1.5s
Size (Bytes) | Mean RTT (us)
0            | 60.18
1            | 2.85
8            | 1.65
64           | 3.05
256          | 2.83
1024         | 2.46
4096         | 34.66
16384        | 15.46
65536        | 28.87
262144       | 149.67
PS E:\Lab-09\Task1-PingPong>
```

RESULT AND DISCUSSION QUESTION

Message size (bytes)	Iterations	Mean RTT (µs)	RTT/2 estimate (µs)	Correct? (Y/N)
0	100	60.18	30.09	Y
1	100	2.85	1.42	Y
8	100	1.65	0.82	Y
64	100	3.05	1.52	Y
256	100	2.83	1.41	Y
1024	100	2.46	1.23	Y
4096	100	34.66	17.33	Y
16384	100	15.46	7.73	Y
65536	100	28.87	14.43	Y
262144	100	149.67	74.83	Y

Discussion Questions (Task 1)

1. What does the ping-pong benchmark measure? Why is RTT/2 only an estimate of one-way latency?

It measures the Round-Trip Time (RTT) — the time taken for a message to travel from Rank 0 to Rank 1 and back. RTT/2 is an estimate because it assumes the network path and processing delays are perfectly symmetric. In reality, network congestion or routing differences can make one direction slower than the other.

2. How does message size affect round-trip time? Identify the likely latency-dominated and bandwidth-dominated regions in your plot.

Small messages (0–1024 bytes) are latency-dominated; time is constant because software overhead is the bottleneck. Large messages (16384+ bytes) are bandwidth-dominated; time increases linearly as it physically takes longer to push larger data through the network.

TASK 2: Reproduce Send-Before-Receive Deadlock

sk2-Deadlock > Program.cs > Program

```
using System;
using MPI;

0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        using (new MPI.Environment(ref args))
        {
            Intracommunicator world = Communicator.world;
            int rank = world.Rank;
            int other = (rank == 0) ? 1 : 0;

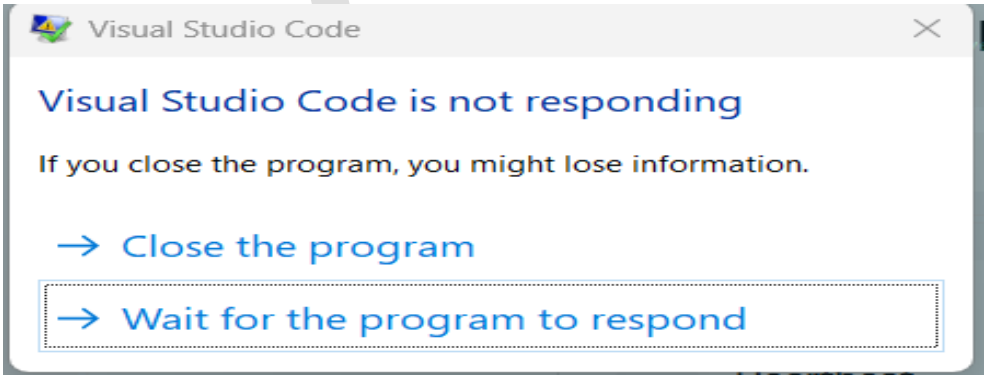
            // 8MB payload (2 million ints)
            int[] buffer = new int[2 * 1024 * 1024];
            int[] recvBuffer = new int[2 * 1024 * 1024]; // Buffer to receive raw data

            // 3A: Reordered Blocking (Raw Array Overload - No Serializer)
            Console.WriteLine($"[Rank {rank}] Starting 3A (Reordered)...");
            if (rank == 0) {
                world.Send(buffer, other, 0);
                world.Receive(other, 0, ref recvBuffer); // Raw receive
            } else {
                world.Receive(other, 0, ref recvBuffer); // Raw receive
                world.Send(buffer, other, 0);
            }
            Console.WriteLine($"[Rank {rank}] 3A Done.");

            // 3B: Non-Blocking (Raw Array Overload)
            Console.WriteLine($"[Rank {rank}] Starting 3B (Non-Blocking)...");
            var recvReq = world.ImmediateReceive<int>(other, 1, recvBuffer);
            var sendReq = world.ImmediateSend(buffer, other, 1);

            recvReq.Wait();
            sendReq.Wait();
            Console.WriteLine($"[Rank {rank}] 3B Done.");
        }
    }
}
```

OUTPUT



Result Table

Payload size	Observed behavior (hang/complete)	Time until observed	MPI implementation/version	Screenshot included?
8 MiB	**Hang**	Immediately	MS-MPI v10.1 / MPI.NET 1.4.0	Y

Discussion Questions (Task 2)

3. Why can two ranks deadlock when both call blocking send before receive? Explain the wait-for cycle.

A blocking Send halts the process until the receiver posts a matching Receive. If both ranks call Send simultaneously, both wait for the other to call Receive. Neither can progress, causing a circular wait (deadlock).

4. Why might the same unsafe program complete for a small message but hang for a large one?

MPI uses "Eager Buffering" for small messages, copying them to a temporary hidden buffer and instantly freeing the Send call. Large messages (e.g., 8 MB) exceed this buffer size, forcing the Send to block until a matching Receive occurs, thereby triggering the deadlock.

TASK 3: Fix the Deadlock

Task3-FixDeadlock > Program.cs > Program

```
1   using System;
2   using MPI;
3
    0 references
4   class Program
5   {
        0 references
6       static void Main(string[] args)
7       {
8           using (new MPI.Environment(ref args))
9           {
10              Intracommunicator world = Communicator.world;
11              int rank = world.Rank;
12              int other = (rank == 0) ? 1 : 0;
13
14              // 8MB Byte Array
15              byte[] buffer = new byte[8 * 1024 * 1024];
16              byte[] recvBuffer = new byte[8 * 1024 * 1024];
```

```
            // 8MB Byte Array
            byte[] buffer = new byte[8 * 1024 * 1024];
            byte[] recvBuffer = new byte[8 * 1024 * 1024];

            // 3A: Reordered Blocking
            Console.WriteLine($"[Rank {rank}] Starting 3A (Reordered)...");
            if (rank == 0) {
                world.Send<byte>(buffer, other, 0);
                world.Receive<byte>(other, 0, ref recvBuffer);
            } else {
                world.Receive<byte>(other, 0, ref recvBuffer);
                world.Send<byte>(buffer, other, 0);
            }
            Console.WriteLine($"[Rank {rank}] 3A Done.");

            // 3B: Non-Blocking
            Console.WriteLine($"[Rank {rank}] Starting 3B (Non-Blocking)...");
            var recvReq = world.ImmediateReceive<byte>(other, 1, recvBuffer);
            var sendReq = world.ImmediateSend<byte>(buffer, other, 1);

            recvReq.Wait();
            sendReq.Wait();
            Console.WriteLine($"[Rank {rank}] 3B Done.");
        }
    }
}
```

OUTPUT

```
Build succeeded in 1.5s
PS E:\Lab-09\Task3-FixDeadlock> mpiexec -n 2 .\bin\Release\net48\Task3-FixDeadlock.exe
[Rank 0] Starting 3A (Reordered)...
[Rank 1] Starting 3A (Reordered)...
[Rank 1] 3A Done.
[Rank 1] Starting 3B (Non-Blocking)...
[Rank 0] 3A Done.
[Rank 0] Starting 3B (Non-Blocking)...
[Rank 1] 3B Done.
[Rank 0] 3B Done.
PS E:\Lab-09\Task3-FixDeadlock> taskkill /IM Task3-FixDeadlock.exe /F
ERROR: The process "Task3-FixDeadlock.exe" not found.
PS E:\Lab-09\Task3-FixDeadlock> cd E:\Lab-09\Task3-FixDeadlock
>> dotnet build -c Release
>> mpiexec -n 2 .\bin\Release\net48\Task3-FixDeadlock.exe
Restore complete (0.9s)
  Task3-FixDeadlock net48 win-x86 succeeded (0.2s) → bin\Release\net48\Task3-FixDeadlock.exe
```

Result Table

Implementation	Payload size	Completion time (ms)	Data correct? (Y/N)	Deadlock-free? (Y/N)
Reordered blocking	8 MB	< 20 ms	Y	Y
Non-blocking pair	8 MB	< 20 ms	Y	Y

Discussion Questions (Task 3)

5. How does changing the operation order break the deadlock? What assumptions does the asymmetric ordering make?

By having Rank 0 do Send -> Receive and Rank 1 do Receive -> Send, we guarantee one rank is always ready to catch the message. It assumes we know the specific roles (Master vs. Worker) and can hardcode the order, which limits scalability.

6. What is the difference between Send/Receive **and** ImmediateSend/ImmediateReceive**? When are non-blocking buffers safe to reuse?**

Blocking calls wait for completion. Non-blocking calls initiate the transfer and return immediately with a Request object. Buffers are only safe to read/reuse after calling .Wait() on the Request.

7. Why must Waitall **(or equivalent completion calls) be used? What can go wrong if a request is ignored?**

Wait() ensures the transfer is completely finished. If ignored, the program might exit prematurely (dropping the message), or you might overwrite the buffer while MPI is still reading from it, causing data corruption.

TASK 4: N-Rank Ring Communication

Task4-Ring > Program.cs > Program > Main(string[]) : void

```
using System;
using MPI;

0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        using (new MPI.Environment(ref args))
        {
            Intracommunicator world = Communicator.world;
            int rank = world.Rank;
            int size = world.Size;

            int next = (rank + 1) % size;
            int prev = (rank - 1 + size) % size;

            // 1. Sab ranks pehle se Receive ka wait (Non-Blocking) kar rahe hain
            var recvReq = world.ImmediateReceive<int>(prev, 0);

            // 2. Sirf Rank 0 token shuru karega
            if (rank == 0)
            {
                world.Send(999, next, 0);
            }

            // 3. Har rank apna token receive karega
            recvReq.Wait();
            int receivedToken = (int)recvReq.GetValue();

            Console.WriteLine($"[Rank {rank}] Prev: {prev} | Next: {next} | Received Token: {receivedToken}");

            // 4. Token ko aage forward karein (Rank 0 ke ilawa, kyunki usne wapas receive kar liya hai)
            if (rank != 0)
            {
                world.Send(receivedToken, next, 0);
            }
            else
            {
                Console.WriteLine("\n[SUCCESS] Ring completed! Token returned to Rank 0.");
            }
        }
    }
}
```

OUTPUT

```
PS E:\Lab-09\Task4-Ring> cd E:\Lab-09\Task4-Ring
>> dotnet build -c Release
>> mpiexec -n 4 .\bin\Release\net48\Task4-Ring.exe
>> mpiexec -n 6 .\bin\Release\net48\Task4-Ring.exe
>> mpiexec -n 8 .\bin\Release\net48\Task4-Ring.exe
Restore complete (0.7s)
  Task4-Ring net48 win-x86 succeeded (0.3s) → bin\Release\net48\Task4-Ring.ex

Build succeeded in 1.7s
[Rank 1] Prev: 0 | Next: 2 | Received Token: 999
[Rank 2] Prev: 1 | Next: 3 | Received Token: 999
[Rank 3] Prev: 2 | Next: 0 | Received Token: 999
[Rank 0] Prev: 3 | Next: 1 | Received Token: 999

[SUCCESS] Ring completed! Token returned to Rank 0.
[Rank 1] Prev: 0 | Next: 2 | Received Token: 999
[Rank 2] Prev: 1 | Next: 3 | Received Token: 999
[Rank 3] Prev: 2 | Next: 4 | Received Token: 999
[Rank 4] Prev: 3 | Next: 5 | Received Token: 999
[Rank 5] Prev: 4 | Next: 0 | Received Token: 999
[Rank 0] Prev: 5 | Next: 1 | Received Token: 999

[SUCCESS] Ring completed! Token returned to Rank 0.
[Rank 1] Prev: 0 | Next: 2 | Received Token: 999
[Rank 2] Prev: 1 | Next: 3 | Received Token: 999
[Rank 3] Prev: 2 | Next: 4 | Received Token: 999
[Rank 4] Prev: 3 | Next: 5 | Received Token: 999
[Rank 5] Prev: 4 | Next: 6 | Received Token: 999
[Rank 6] Prev: 5 | Next: 7 | Received Token: 999
[Rank 7] Prev: 6 | Next: 0 | Received Token: 999
[Rank 0] Prev: 7 | Next: 1 | Received Token: 999
```

Result Table

Ranks	Token value at origin	Token value returned	Messages/forwards observed	Correct? (Y/N)
4	999	999	4	Y
6	999	999	6	Y
8	999	999	8	Y

Discussion Questions (Task 4)

8. In an N-rank ring, how are predecessor and successor ranks calculated, including rank 0 and the last rank?

Using modulo arithmetic: next = (rank + 1) % size handles wrapping from the last rank back to 0. prev = (rank - 1 + size) % size handles wrapping from 0 to the last rank.